JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang.invoke

Class SerializedLambda

java.lang.Object
    java.lang.invoke.SerializedLambda

All Implemented Interfaces:

Serializable

```
public final class SerializedLambda
extends Object
implements Serializable
```

Serialized form of a lambda expression. The properties of this class represent the information that is present at the lambda factory site, including static metafactory arguments such as the identity of the primary functional interface method and the identity of the implementation method, as well as dynamic metafactory arguments such as values captured from the lexical scope at the time of lambda capture.

Implementors of serializable lambdas, such as compilers or language runtime libraries, are expected to ensure that instances deserialize properly. One means to do so is to ensure that the `writeReplace` method returns an instance of `SerializedLambda`, rather than allowing default serialization to proceed.

`SerializedLambda` has a `readResolve` method that looks for a (possibly private) static method called `$deserializeLambda$(SerializedLambda)` in the capturing class, invokes that with itself as the first argument, and returns the result. Lambda classes implementing `$deserializeLambda$` are responsible for validating that the properties of the `SerializedLambda` are consistent with a lambda actually captured by that class.

See Also:

LambdaMetafactory, Serialized Form

Constructor Summary

Constructors

Constructor
`SerializedLambda(Class<?> capturingClass, String functionalInterfaceClass, String`

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`Object`	`getCapturedArg(int i)`	Get a dynamic argument to the lambda capture site.
`int`	`getCapturedArgCount()`	Get the count of dynamic arguments to the lambda capture site.
`String`	`getCapturingClass()`	Get the name of the class that captured this lambda.
`String`	`getFunctionalInterfaceClass()`	Get the name of the invoked type to which this lambda has been converted
`String`	`getFunctionalInterfaceMethodName()`	Get the name of the primary method for the functional interface to which this lambda has been converted.

String	getFunctionalInterfaceMethodSignature()	Get the signature of the primary method for the functional interface to which this lambda has been converted.
String	getImplClass()	Get the name of the class containing the implementation method.
int	getImplMethodKind()	Get the method handle kind (see MethodHandleInfo) of the implementation method.
String	getImplMethodName()	Get the name of the implementation method.
String	getImplMethodSignature()	Get the signature of the implementation method.
String	getInstantiatedMethodType()	Get the signature of the primary functional interface method after type variables are substituted with their instantiation from the capture site.
String	toString()	Returns a string representation of the object.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Detail

SerializedLambda

```
public SerializedLambda(Class<?> capturingClass,
                        String functionalInterfaceClass,
                        String functionalInterfaceMethodName,
                        String functionalInterfaceMethodSignature,
                        int implMethodKind,
                        String implClass,
                        String implMethodName,
                        String implMethodSignature,
                        String instantiatedMethodType,
                        Object[] capturedArgs)
```

Create a `SerializedLambda` from the low-level information present at the lambda factory site.

Parameters:

`capturingClass` - The class in which the lambda expression appears

`functionalInterfaceClass` - Name, in slash-delimited form, of static type of the returned lambda object

`functionalInterfaceMethodName` - Name of the functional interface method for the present at the lambda factory site

`functionalInterfaceMethodSignature` - Signature of the functional interface method present at the lambda factory site

`implMethodKind` - Method handle kind for the implementation method

`implClass` - Name, in slash-delimited form, for the class holding the implementation method

`implMethodName` - Name of the implementation method

`implMethodSignature` - Signature of the implementation method

`instantiatedMethodType` - The signature of the primary functional interface method after type variables are substituted with their instantiation from the capture site

`capturedArgs` - The dynamic arguments to the lambda factory site, which represent variables captured by the lambda

Method Detail

getCapturingClass

```
public String getCapturingClass()
```

Get the name of the class that captured this lambda.

Returns:

the name of the class that captured this lambda

getFunctionalInterfaceClass

```
public String getFunctionalInterfaceClass()
```

Get the name of the invoked type to which this lambda has been converted

Returns:

the name of the functional interface class to which this lambda has been converted

getFunctionalInterfaceMethodName

```
public String getFunctionalInterfaceMethodName()
```

Get the name of the primary method for the functional interface to which this lambda has been converted.

Returns:

the name of the primary methods of the functional interface

getFunctionalInterfaceMethodSignature

```
public String getFunctionalInterfaceMethodSignature()
```

Get the signature of the primary method for the functional interface to which this lambda has been converted.

Returns:

the signature of the primary method of the functional interface

getImplClass

```
public String getImplClass()
```

Get the name of the class containing the implementation method.

Returns:

the name of the class containing the implementation method

getImplMethodName

```
public String getImplMethodName()
```

Get the name of the implementation method.

Returns:

the name of the implementation method

getImplMethodSignature

```
public String getImplMethodSignature()
```

Get the signature of the implementation method.

Returns:

the signature of the implementation method

getImplMethodKind

```
public int getImplMethodKind()
```

Get the method handle kind (see MethodHandleInfo) of the implementation method.

Returns:

the method handle kind of the implementation method

getInstantiatedMethodType

```
public final String getInstantiatedMethodType()
```

Get the signature of the primary functional interface method after type variables are substituted with their instantiation from the capture site.

Returns:

the signature of the primary functional interface method after type variable processing

getCapturedArgCount

```
public int getCapturedArgCount()
```

Get the count of dynamic arguments to the lambda capture site.

Returns:

the count of dynamic arguments to the lambda capture site

getCapturedArg

```
public Object getCapturedArg(int i)
```

Get a dynamic argument to the lambda capture site.

Parameters:

i - the argument to capture

Returns:

a dynamic argument to the lambda capture site

toString

```
public String toString()
```

Description copied from class: Object

Returns a string representation of the object. In general, the `toString` method returns a string that "textually represents" this object. The result should be a concise but informative representation that is easy for a person to read. It is recommended that all subclasses override this method.

The `toString` method for class `Object` returns a string consisting of the name of the class of which the object is an instance, the at-sign character `@', and the unsigned hexadecimal representation of the hash code of the object. In other words, this method returns a string equal to the value of:

```
getClass().getName() + '@' + Integer.toHexString(hashCode())
```

Overrides:

`toString` in class `Object`

Returns:

a string representation of the object.

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.